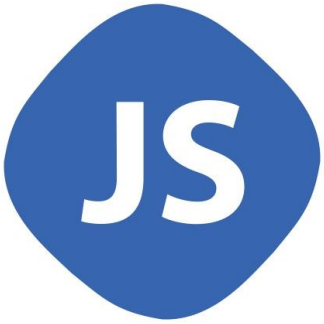


Область видимости и лексическое окружение

- Введение
- Объявление переменных
- Области видимости
- Лексическое окружение для области видимости
- Цепочки объектов лексического окружения
- Заключение

Введение

В текущем уроке рассмотрим важную взаимосвязь понятий – **область видимости** и **лексическое окружение для области видимости**. Понятия очень близкие, но это не одно и тоже.

> ***Примечание***. Область видимости мы уже рассматривали, но с изучением функций сделаем это еще раз более подробно. И тем не менее, к области видимости вернемся еще раз, при изучении темы: **Объектное программирование**.

На первом этапе изучения полезно представить эти понятия в следующих определениях:

- **Область видимости** представляет собой определенную **область программы**, в пределах которой установлена связь между некоторой переменной и её идентификатором.
- **Лексическое окружение для области видимости** — это доступная из **текущей** области видимости **структура данных** (**объект**), в которой хранятся связи между идентификаторами и переменными из всех **внешних областей видимости**. Для каждой области видимости





создается **своё лексическое окружение**. Лексическое окружение **меняется** в ходе выполнения программы.

Если то же самое, но в более простой форме:

- **Область видимости** – **область программы**, в пределах которой имеется доступ к определенным в этой области переменным.
- **Лексическое окружение** – **структура данных** (объект), создаваемый для хранения переменных и их значений в определенной области видимости.

Или еще короче:

- **Область видимости** – **область** программы.
- **Лексическое окружение** – **объект**.

> ***Примечание***. В разделе "Область видимости" важно понять, **как** ведут себя разные переменные в глобальной и локальной области видимости. В разделе "Лексическое окружение для области видимости" – **почему** они себя ведут так или иначе.

Объявление переменных

Переменная – это именованное хранилище для данных. В переменных можно хранить как простые (скалярные), так и структурированные типы данных (массивы, объекты). В ходе выполнения программы значение переменной можно перезаписать, при этом переменной можно присвоить данные нового типа.

Для создания переменной в JavaScript используют следующие ключевые слова:

- **let**
- **const**
- **var**
- переменная **без использования** ключевых слов



Переменные, объявленные **без использования** ключевых слов, будут вести себя в зависимости от места их объявления.

Ключевое слово let / const

Ключевое слово **let** — объявляет переменную, которой можно присвоить новое значение после объявления, то есть переопределить.

Ключевое слово **const** — так же объявляет переменную, но с одним отличием: объявляемая переменная должна быть немедленно инициализирована значением и этой переменной не может быть присвоено другое значение после инициализации, попытка сделать это приведёт к ошибке. **Переменные**, объявленные с помощью **const**, называются **константами**. Названия констант, как правило, пишутся с использованием заглавных букв и знаков подчёркивания.

Объявление **const** делает код более читаемым, так как переменная всегда ссылается на одно и то же значение. В случае с **let** такой уверенности нет.

Переменные и константы, объявленные через ключевые слова **let** и **const** ведут себя одинаково по отношению к лексическому окружению, области видимости.

> ***Важно***. Переменные и константы с ключевым словом **let** / **const не создают свойств** на глобальном объекте **window**.

> ***Примечание***. Поведение **переменных** и **констант**, объявленных через ключевые **let** / **const** предсказуемо. Код современных программ предпочтительно писать именно через эти ключевые слова.

Ключевое слово var



Ключевое слово **var** в JavaScript используется для объявления переменной и **опционального** присвоения ей значения. Var - сокращение от слова "variable" - переменная, **переменная** величина.

Для var не существует блочной области видимости. Область видимости переменных **var** ограничивается либо функцией, либо, если переменная глобальная, то скриптом. Такие переменные доступны за пределами блока.

Для переменных var допускает повторное объявление. Если в блоке кода дважды объявить одну и ту же переменную **let**, будет ошибка:

```
let user;

let user; // SyntaxError: Identifier 'user' has already been declared
```

Используя **var**, можно переобъявлять переменную сколько угодно раз. Повторное использование **var** игнорируются:

```
var user = 'master';

var user = 'admin';

console.log(user); // admin
```

Объявления переменных var обрабатываются в начале выполнения функции или **запуска скрипта**, если переменная является глобальной.

Другими словами, переменные **var** считаются объявленными **с самого начала исполнения** функции/скрипта вне зависимости от того, в каком месте реально находятся их объявления (при условии, что они не находятся во вложенной функции).

Приведенные ниже фрагменты кода полностью эквиваленты:

```
// классическое расположение кода в теле функции

function sayHi() {

    var msg;

    msg = "Привет";
```

```
    alert(msg);
}

// альтернативное расположение кода в теле функции
function sayHi() {
    msg = "Привет";
    alert(msg);
    var msg;
}
```

> ***Примечание.*** Поведение переменных **var** когда объявления всплывают в самый верх функции, называется **hoisting** (всплытие, поднятие).

Стоит обратить внимание, **объявления** переменных всплывают, но **присваивания значений – нет**. Это проще всего продемонстрировать на примере:

```
function sayHi() {
    console.log(msg); // undefined
    msg = "Привет";
    var msg;
    console.log(msg); // Привет
}
```

Поскольку все объявления переменных **var** обрабатываются в начале функции, мы можем ссылаться на них в любом месте. Однако, переменные имеют значение **undefined** до строки с присвоением значения.

> ***Примечание***. Поведение **переменных**, объявленных через ключевое слово **var** вычислить можно, но использование таких переменных нежелательно.



Переменная без let / const / var

Если **var** опущено, то переменная, которой присваивается значение, автоматически становится глобальной, независимо от того, где она находится в коде. Это может привести к нежелательному поведению, если одна и та же переменная используется в разных частях кода для разных целей.

> ***Примечание***. Писать код с **переменными, объявленными без использования ключевых** слов можно, если вам безразлично структурирование и поддержка кода. В реальных приложениях использовать такой стиль крайне нежелательно.

Далее в многочисленных примерах и рисунках разбираем заявленные вопросы урока.

Область видимости

Область видимости – это некоторая сущность JavaScript, которая определяет границы действия переменных. Создаются области видимости во время **выполнения программы**.

> ***Примечание***. Самая первая область, которая создаётся и которая включает в себя все остальные называется **глобальной областью видимости**.

Именно в глобальной области видимости веб-браузера определена переменная **window**.

example_1.

```
<script>
    // глобальный объект window присутствует в сценарии всегда
    // содержимое объекта всегда можно просмотреть
```

```
    console.log(window);
</script>
```

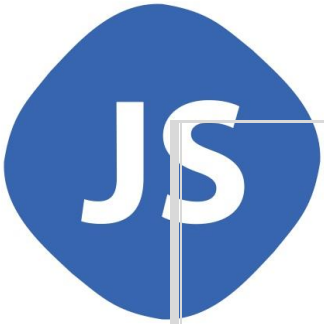
Программист может определять пользовательские переменные в этой области. Для этого достаточно просто объявить переменные вне **блока**, **функции** и **модуля**. В этом случае они будут находиться в **глобальной области видимости**.

Глобальная область видимости

Переменные, объявленные в глобальной области видимости называются **глобальными переменными**. Глобальные переменные доступны в любой точке программы.

example_2.

```
<script>
    // ГЛОБАЛЬНАЯ переменная с записью в window
    var aVarGlobal = 'Var Global variable';
    // ГЛОБАЛЬНАЯ переменная без записи в window
    let aLetGlobal = 'Let Global variable';
    // ГЛОБАЛЬНАЯ константа без записи в window
    const aConstGlobal = 'This is a constant';
    // ГЛОБАЛЬНАЯ переменная с записью в window
    // без ключевого слова var/let
    aGlobal = 'Global variable';
    // определяем функцию
    // функция создает локальную область видимости
    function aOutVariable() {
        console.log('Вывод ГЛОБАЛЬНЫХ переменных из ЛОКАЛЬНОЙ
```



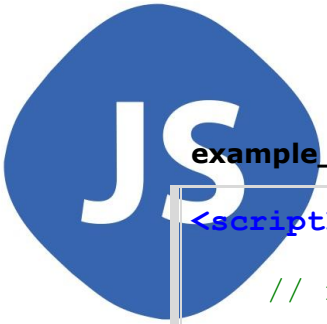
```
        области видимости:');
        console.log(aVarGlobal);  // Var Global variable
        console.log(aLetGlobal);  // Let Global variable
        console.log(aConstGlobal);  // This is a constant
        console.log(aGlobal);  // Global variable
    }
    // вызываем функцию
    aOutVariable();
    console.log('Вывод ГЛОБАЛЬНЫХ переменных из ГЛОБАЛЬНОЙ
    области видимости:');
    console.log(aVarGlobal);  // Var Global variable
    console.log(aLetGlobal);  // Let Global variable
    console.log(aConstGlobal);  // This is a constant
    console.log(aGlobal);  // Global variable
    // ----------
    console.log(window);
</script>
```

Обратите внимание, что сама функция также присутствует в свойствах объекта **window**. Значение свойства – определение функции.

> ***Примечание***. Браузеры по-разному отображают объект **window** в консоли. Пользовательский префикс "**a**" в именах переменных и функций гарантированно выводит создаваемые свойства и функции объекта в начало списка.

Так как к глобальным переменным есть доступ из локальной области видимости, переопределить такую переменную из функции не составит труда.





example_3.

```
<script>
    // глобальная переменная с записью в window
    var aVarGlobal = "Var Global variable";
    // глобальная переменная без записи в window
    let aLetGlobal = 'Let Global variable';
    // глобальная переменная с записью в window
    // без ключевого слова var/let
    aGlobal = 'Global variable';
    // определяем функцию
    function aReVariable() {
        // переопределяем глобальную переменную var
        aVarGlobal = "Alea jacta est";
        // переопределяем глобальную переменную let
        aLetGlobal = "Fiat lux!";
        // переопределяем глобальную переменную без var/let
        aGlobal = "Acta est fabŭla!";
    }
    // вызываем функцию
    aReVariable();
    // выводим новые значения глобальных переменных
    console.log(aVarGlobal);  // Alea jacta est
    console.log(aLetGlobal);  // Fiat lux!
    console.log(aGlobal);  // Acta est fabŭla
    // ----------
    console.log(window);
```



```
</script>
```

Из локальной области видимости можно не только переопределить, но и создать глобальную переменную. Почему так происходит разберем в разделе **Цепочки областей видимости**.

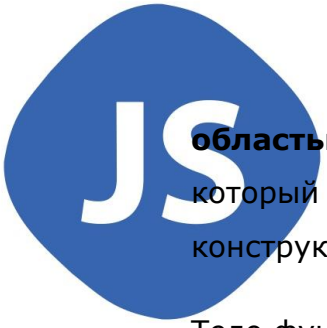
example_4.

```
<script>
    // определяем функцию выражение
    aCreateGlobalVariable = function () {
        // создаем глобальную переменную
        aGlobalVar = "I'm Global variable";
    }
    // вызываем функцию
    aCreateGlobalVariable();
    // проверяем наличие глобальной переменной
    console.log(aGlobalVar);
    // ----------
    console.log(window);
</script>
```

Локальная область видимости

Кроме глобальной области видимости в JavaScript имеется **локальная область видимости**. Переменные, объявленные в локальной области видимости называются **локальными**. Локальные переменные имеют область видимости, ограниченную блоком. Т.е. они привязаны к нему и будут актуальны только в его рамках.

Локальная область создается, когда интерпретатор выполняет код функции или блочной конструкции. Такая локальная область видимости называется



областью видимости блока. Под блоком в JavaScript понимается любой код, который расположен в **фигурных скобках {...}**. Блоки используются в конструкциях **if**, **switch**, **for**, **while** и т.д.

Тело функции тоже является блоком, т.к. находится между фигурными скобками. Вызов функции создает **область видимости функции**.

Кроме этого, локальная область видимости создается вызовами **модулей**. Такая область называется **областью видимости модуля**.

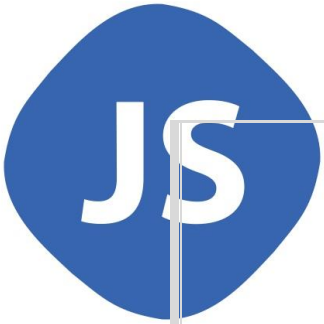
В следующих демонстрационных примерах к глобальной области видимости будем добавлять функциональную область видимости, созданную разными конструкциями языка программирования JavaScript.

Локальная область видимости

Область видимости блока (скобок)

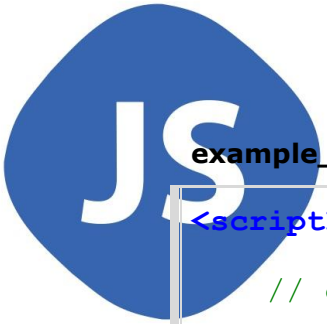
example_5.

```
<script>
    // глобальная область видимости
    // глобальные переменные
    var aVarVariable = 'Global variable Var';
    let aLetVariable = 'Global variable Let';
    aVariable = 'Global variable';
    // область видимости блока (скобок)
    {
        // переменные уровня блока
        // создаем переменные с именами глобальных переменных
        // локальная переменная var
```



```
        // var-переменная НЕ ЗНАЕТ о существовании блока
        var aVarVariable = null;
        // локальная переменная let
        // let-переменная ЗНАЕТ о существовании блока
        let aLetVariable =  null;
        // локальная переменная без var/let
        // переменная НЕ ЗНАЕТ о существовании блока
        aVariable = null;
        console.log('блоковая область видимости');
        // выведем значения переменных после блока
        console.log(aVarVariable);  // null
        console.log(aLetVariable);  // null
        console.log(aVariable);  // null
    }
    console.log('глобальная область видимости');
    // выведем значения переменных после блока
    console.log(aVarVariable);  // null
    console.log(aLetVariable);  // Global variable Let
    console.log(aVariable);  // null
    // ----------
    console.log(window);
</script>
```

Область видимости блока if



example_6.

```
<script>
    // область видимости блока if
    if (true) {
        // локальная переменная var
        // var-переменная НЕ ЗНАЕТ о существовании блока
        var aVarLocal = "I'm a Global Var variable?";
        // локальная переменная let
        // let-переменная ЗНАЕТ о существовании блока
        let aLetLocal = "I'm a Local Let variable!";
        // локальная переменная без var/let
        // переменная НЕ ЗНАЕТ о существовании блока
        aLocal = "Who am I?";
        // блоковая область видимости
        console.log(aVarLocal); // I'm a Global Var variable
        console.log(aLetLocal); // I'm a Local Let variable
        console.log(aLocal); // Who am I?
    }
    console.log('----------');
    // тестируем доступ из глобальной области к переменным
    созданным в локальной области
    console.log(aVarLocal); // I'm a Global Var variable
    // Uncaught ReferenceError: aLetLocal is not defined
    // console.log(aLetLocal); // Error
    console.log(aLocal); // Who am I?
    // ----------
```



```
    console.log(window);
</script>
```

Область видимости цикла

example_7.

```
<script>
    // область видимости цикла (любого)
    for (var ak = 0; ak < 10; ak++) {
        // локальная переменная var
        // var-переменная НЕ ЗНАЕТ о существовании блока
        var aVarLocal = ak + 1;
        // локальная переменная let
        // let-переменная ЗНАЕТ о существовании блока
        let aLetLocal = ak + 1;
        // локальная переменная без var/let
        // переменная НЕ ЗНАЕТ о существовании блока
        aLocal = ak - 1;
    }
    // тестируем доступ к переменным созданным в локальной
    области
    console.log(aVarLocal); // 10
    // Uncaught ReferenceError: aLetLocal is not defined
    // console.log(aLetLocal); // Error
    console.log(aLocal); // 8
    // ----------
    console.log(window);
```

```
</script>
```

Область видимости функции

example_8.

```
<script>

    // область видимости функции

    // определяем функцию инструкцию

    function reVariable() {

        // локальная переменная var

        // var-переменная ЗНАЕТ о существовании блока

        var aVarLocal = "Var Local variable";

        // локальная переменная let

        // let-переменная ЗНАЕТ о существовании блока

        let aLetLocal = "Let Local variable";

        // локальная переменная без var/let

        // переменная НЕ ЗНАЕТ о существовании блока

        aGlobal = "I'm Global variable";

        // блоковая область видимости

        console.log("Локальные переменные уровня функции:");

        console.log(aVarLocal);  // Var Local variable

        console.log(aLetLocal);  // Let Local variable

        console.log(aGlobal);  // I'm Global variable

    }

    // вызов функции

    reVariable();

    console.log("Доступ к локальным переменным из глобальной
```

```
    области:");
    // Uncaught ReferenceError: aVarLocal is not defined
    // console.log(aVarLocal);  // Error
    // Uncaught ReferenceError: aLetLocal is not defined
    // console.log(aLetLocal);  // Error
    console.log(aGlobal);  // I'm Global variable
    // ----------
    console.log(window);
</script>
```

example_9.

```
<script>
    // область видимости функции
    // определяем функцию выражение
    reVariable = function () {
        // локальная переменная var
        // var-переменная ЗНАЕТ о существовании блока
        var aVarLocal = "Var Local variable";
        // локальная переменная let
        // let-переменная ЗНАЕТ о существовании блока
        let aLetLocal = "Let Local variable";
        // локальная переменная без var/let
        // такая переменная НЕ ЗНАЕТ о существовании блока
        aGlobal = "I'm Global variable";
        // блоковая область видимости
        console.log("Локальные переменные уровня функции:");
```

```
        console.log(aVarLocal);  // Var Local variable
        console.log(aLetLocal);  // Let Local variable
        console.log(aGlobal);  // I'm Global variable
    }
    // вызов функции
    reVariable();
    console.log("Доступ к локальным переменным из глобальной области:");
    // Uncaught ReferenceError: aVarLocal is not defined
    // console.log(aVarLocal);  // Error
    // Uncaught ReferenceError: aLetLocal is not defined
    // console.log(aLetLocal);  // Error
    console.log(aGlobal);  // I'm Global variable
    // ----------
    console.log(window);
</script>
```

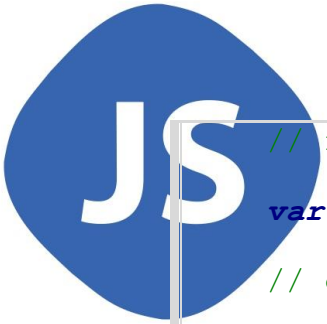
Таким образом, поведение переменных можно описать следующим образом:

- **let**-переменная всегда внутри области видимости, в которой определена;
- **var**-переменная всегда поднимается в глобальную область видимости, за исключением функции;
- переменная без **var**/**let** всегда поднимается в глобальную область видимости.

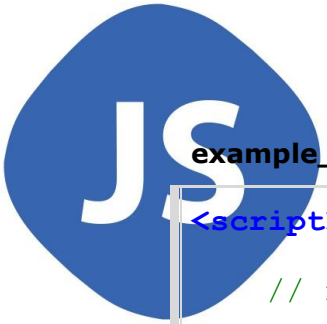
Область видимости объекта

example_10.

```
<script>
```



```
    // глобальное свойство
    var aVariable = "I'm global variable";
    // область видимости объекта
    // определяем глобальный объект со свойством и методом
    var aObj = {
        // свойство объекта
        // не стоит путать переменную и свойство объекта
        aVariable: "I'm property of object",
        // метод объекта
        aMethodObj: function() {
            // что выводим, глобальное свойство или свойство
            объекта?
            console.log(aVariable);
        }
    }
    // глобальная переменная
    console.log(aVariable);  // I'm global variable
    // свойство переменной aObj
    console.log(aObj.aVariable);  // I'm property of object
    // метод переменной aObj
    aObj.aMethodObj();  // I'm global variable
    // ----------
    console.log(window);
</script>
```



example_11.

```
<script>
    // глобальное свойство
    var aVariable = "I'm global variable";
    // область видимости объекта
    var aObj = {
        aVariable: "I'm property of object",
        // метод объекта
        aMethodObj: function() {
            // что переопределили, глобальное свойство или
            свойство объекта?
            aVariable = "А меня переопределили ... :(";
        }
    }
    // глобальная переменная
    console.log(aVariable);  // I'm global variable
    // метод переменной aObj
    aObj.aMethodObj();
    // глобальная переменная
    console.log(aVariable);  // А меня переопределили ... :(
    // ----------
    console.log(window);
</script>
```





Лексическое окружение для области видимости

В предыдущем разделе мы разобрали такие понятия, как границы действия и область видимости переменных. Теперь время разобрать причины, обуславливающие определенное поведение переменных.

В JavaScript каждому выполняемому скрипту соответствует связанный с ними **внутренний** (скрытый) **объект**, называемый **лексическим окружением LexicalEnvironment**.

Лексическое окружение — это объект **LexicalEnvironment**, содержащий набор переменных, функций и объектов доступных в определенной **области видимости**. Объем лексического окружения определяется расположением кода в программе.

Объект **лексического окружения** (**LexicalEnvironment**) состоит из двух частей:

- **Environment Record** – объект, в котором как свойства хранятся все локальные переменные (именно в этом объекте определена **this**).
- **Ссылка на внешнее лексическое окружение** (**outher**) – то есть то, которое соответствует коду снаружи (снаружи от текущих фигурных скобок).

Переменная или **функция** – это просто **свойство** специального внутреннего объекта – **Environment Record**. Получить или изменить переменную, означает, получить или изменить свойство этого объекта.

Самым верхним лексическим окружением является объект **лексического окружения** для **глобальной области видимости**. Это, так называемое, глобальное лексическое окружение, связанное со всем скриптом.

Глобальное окружение может содержать **вложенные лексические окружения,** определяемые конструкциями языка программирования JavaScript содержащими **скобочные группы**.

Важно. Во время выполнения программы на JavaScript, для каждой скобочной группы создается **новая область видимости** и, соответственно, **новое лексическое окружение**, которое содержит все переменные, функции и объекты, определенные внутри этой скобочной группы.

Под скобочной группой понимается любой код блочной конструкции, расположенной в **фигурных скобках {...}**. Блоки используются в конструкциях **if**, **switch**, **for**, **while** и т.д.

Пример 1.

```
// переменная глобальной области видимости
let a = 5;
{
    let b = 17;
    console.log(b); // 17
}
console.log(a); // 5
console.log(b); // Uncaught ReferenceError: b is not defined
```

Представленный фрагмент программы содержит два объекта лексического окружения под каждую из областей видимости.

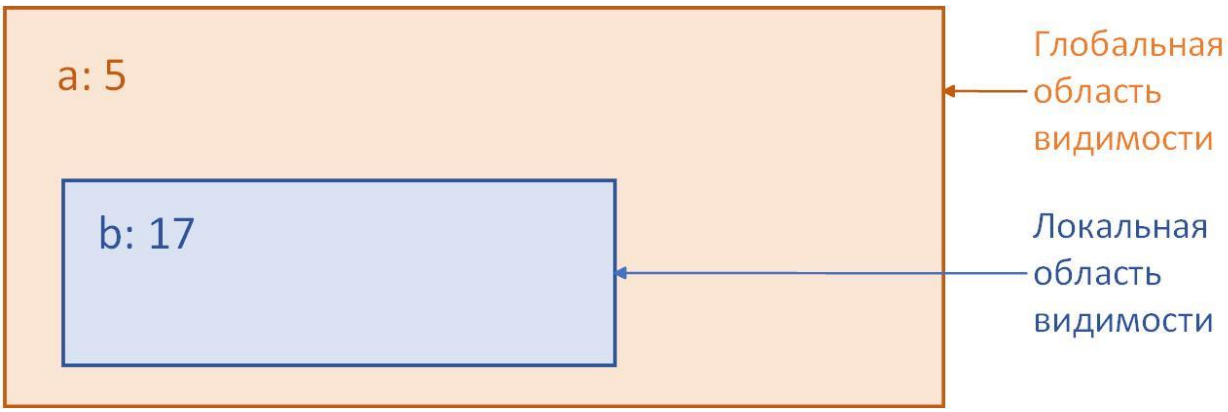


Рис.1. Глобальный и локальный объекты лексические окружения

Пример 2.

```
// определение глобальной функции
function salute(welcomeText) {
      console.log(welcomeText);
      alert(welcomeText);
}
// первый вызов функции
salute('Привет');
// второй вызов функции
salute('Здравствуйте');
```

Фрагмент содержит глобальный лексический объект и два локальных объекта, созданных под каждый из вызовов функции.

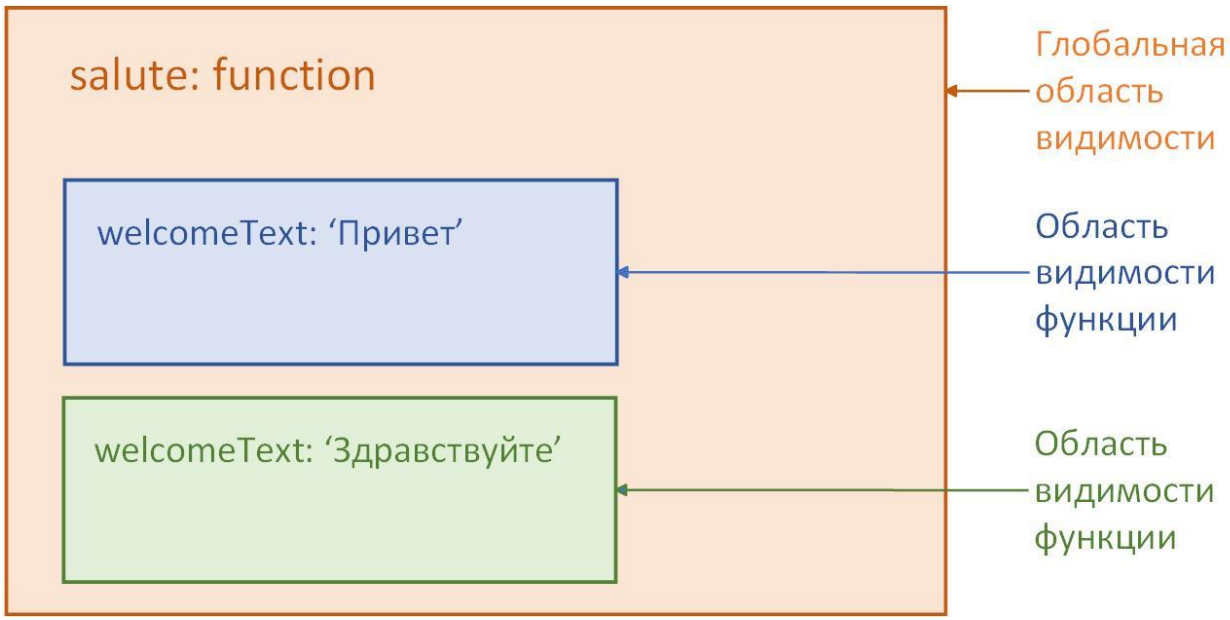
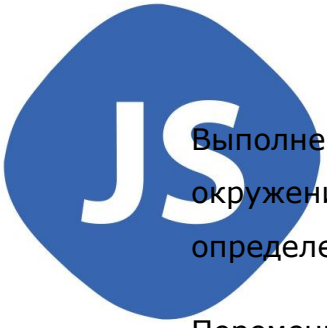


Рис.2. Последовательное создание объектов лексического окружения

Лексическое окружение – это объект спецификации: он существует только теоретически в спецификации языка для описания того, как все работает. Мы не можем получить этот объект в нашем коде и манипулировать им напрямую.

Глобальное лексическое окружение



Выполнение программы начинается с создания глобального лексического окружения, **содержащего все переменные, функции и объекты**, определенные вне какой-либо скобочной группы.

Переменные и функции, определенные в глобальной лексической среде, доступны для всего кода, выполняемого в программе. У глобального лексического окружения **нет внешнего окружения**, таким образом, ссылка на внешнее лексическое окружение указывает на **null**.

> ***Важно***. По мере выполнения кода **лексическое окружение меняется**.

В демонстрационном примере **example_12** продемонстрировано изменение глобального лексического окружения.

1. Объявление let-переменной **msg** – глобальной области видимости.
2. Инициализация переменной **msg** текстовым значением "Добро пожаловать!!".

example_12.

```
<script>
    // глобальное лексическое окружение
    // 1
    let msg;
    // 2
    msg = 'Добро пожаловать!';
    // ----------
    console.log(window);
</script>
```

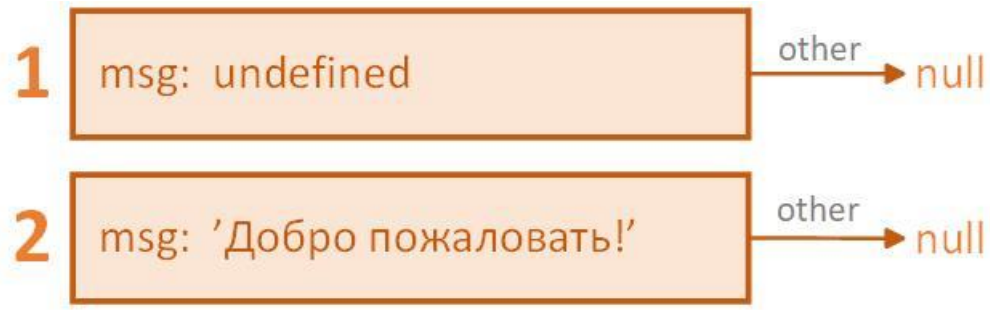


Рис.3. Формирование глобального лексического окружения

В демонстрационном примере **example_13** продолжаем работать с глобальным лексическим окружением.

1. Объявление let-переменной **msg** – глобальной области видимости.
2. Инициализация переменной **msg** текстовым значением "Добро пожаловать!".
3. Переопределение переменной **msg**.

example_13.

```
<script>
    // глобальное лексическое окружение
    // 1
    let msg;
    // 2
    msg = 'Добро пожаловать!';
    // 3
    msg = 'Посторонним В.';
    // ----------
    console.log(window);
</script>
```

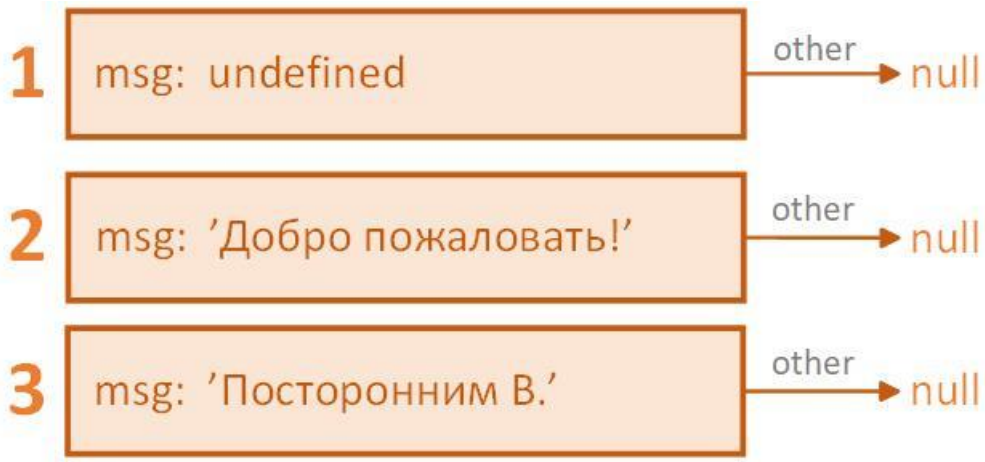


Рис.4. Формирование глобального лексического окружения

В демонстрационном примере **example_14** к глобальному лексическому окружению добавим новое свойство, содержащее определение функции **fnAuth**().

1. Объявление let-переменной **msg** – глобальной области видимости.
2. Инициализация переменной **msg** текстовым значением "Добро пожаловать!".
3. Переопределение переменной **msg**.
4. Добавление к глобальному лексическому окружению определения функции **fnAuth**().

Обратите внимание, никакого вложенного окружения пока не существует.

example_14.

```
<script>

    // глобальное лексическое окружение

    // 1

    let msg;

    // 2

    msg = 'Добро пожаловать!';

    // 3

    msg = 'Посторонним В.';
```

```
    // 4
    function fnAuth(login, pwd) {
        // область видимости функции
        // вызов функции создаст локальное (вложенное)
        лексическое окружение
        var email = 'd-dm@mail.ru';
        /*  локальные переменные функции:
            -- login
            -- pwd
            -- email
        */
    }
    // ----------
    console.log(window);
</script>
```

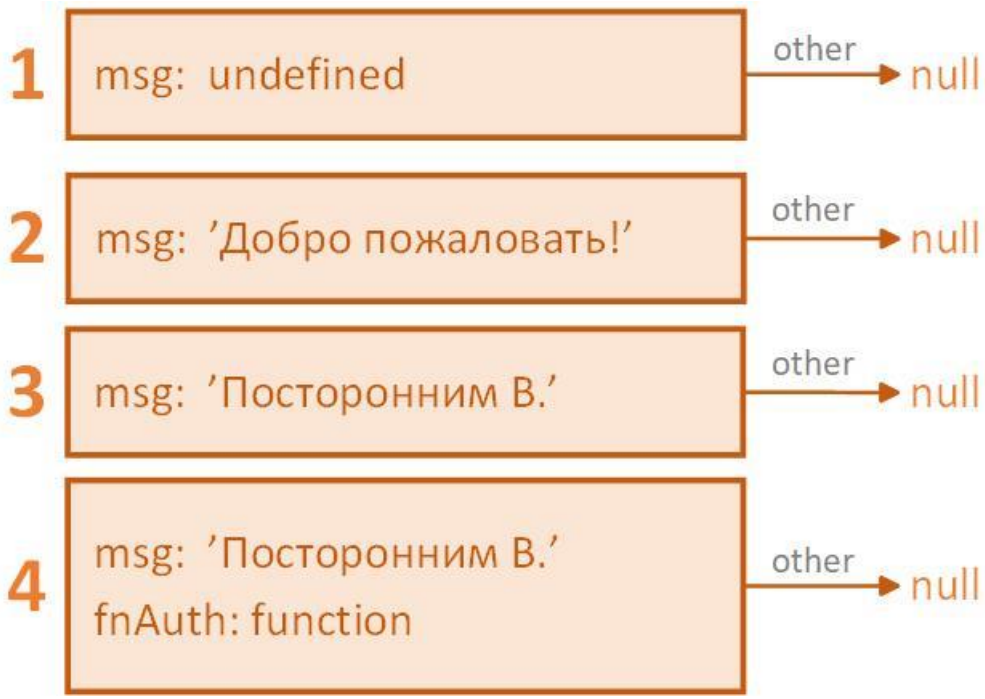




Рис.5. Формирование глобального лексического окружения

Локальное лексическое окружение

JavaScript поддерживает создание вложенных лексических сред окружения.

Каждый выполняемый **блок кода {**...**}** (function, if, switch, while ...) создает свой собственный внутренний (скрытый) объект лексического окружения **LexicalEnvironment**.

В демонстрационном примере **example_15** к глобальному лексическому окружению добавляется вложенное.

1. Объявление let-переменной **msg** – глобальной области видимости.
2. Инициализация переменной **msg** текстовым значением "Добро пожаловать!".
3. Переопределение переменной **msg**.
4. Добавление к глобальному лексическому окружению определения функции **fnAuth**().
5. Создание вложенного лексического окружения (объекта Environment Record) при вызове функции **fnAuth**(). Вложенный объект содержит набор принадлежащих ему локальных переменных.

Обратите внимание, вложенный объект имеет ссылку на объект родителя, в данном примере родителем является объект **Environment Record** глобального лексического окружения.

example_15.

```
<script>

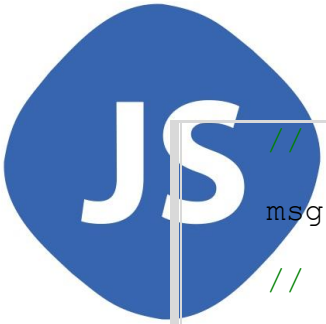
    // глобальное лексическое окружение

    // 1

    let msg;

    // 2

    let msg = 'Добро пожаловать!';
```



```
    // 3
    msg = 'Посторонним В.';
    // 4
    function fnAuth(login, pwd) {
        // область видимости функции
        // локальное (вложенное) лексическое окружение
        var email = 'w-t-p@mail.ru';
        /*  локальные переменные функции:
            -- login
            -- pwd
            -- email
        */
        // console.log(login, pwd, email)
    }
    // 5
    // вызов функции создает локальное лексическое окружение
    // аргументы функции попадут в свойства локального
    лексического объекта
    fnAuth('Winnie-the-Pooh', 'ilovehoney');
    // ----------
    console.log(window);
</script>
```

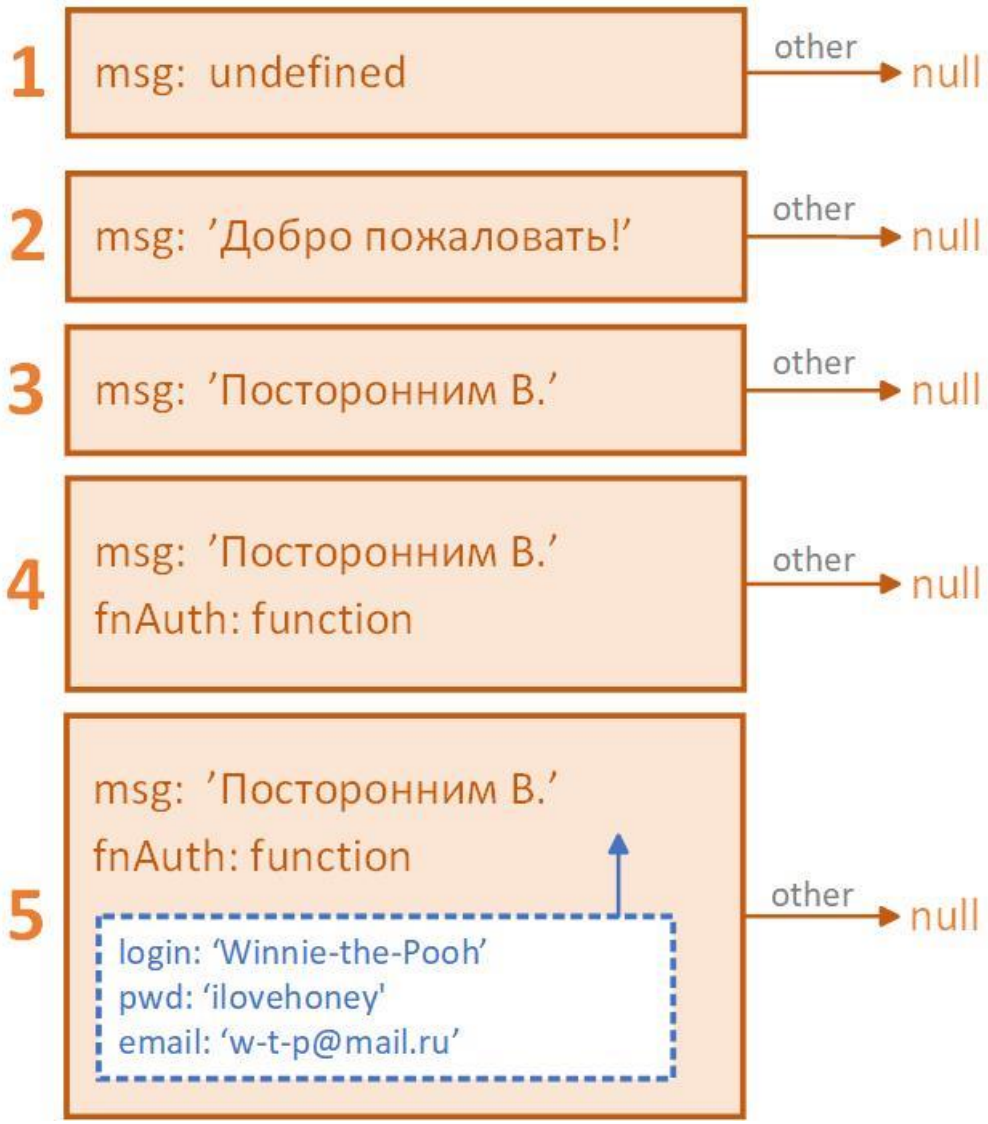
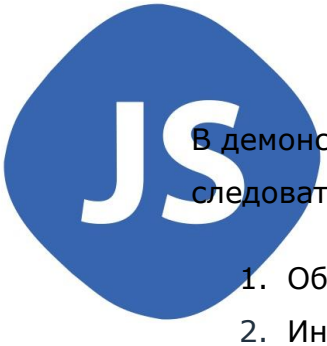


Рис.6. Формирование глобального и локального лексических окружений

Когда **запускается** (запускается, а не определяется) функция, в начале ее вызова автоматически создается новое лексическое окружение для хранения **локальных переменных** и **параметров вызова**. В процессе вызова функции у нее есть два лексических окружения:

- **внутреннее** (для вызываемой функции). Внутреннее лексическое окружение содержит свойства **login**, **pwd**, **email** и **ссылку** на внешнее окружение (outer).
- **внешнее** (глобальное). Содержит свойство **msg** и определение самой функции **fnAuth**(). Ссылка на внешнее окружение для глобальной области – **null**.



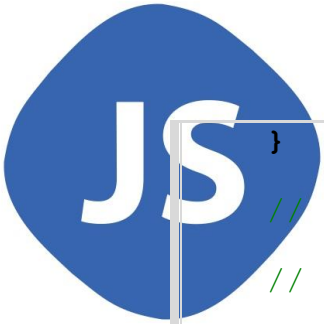
В демонстрационном примере **example_16** функция вызывается дважды, следовательно вложенный лексический объект будет создаваться два раза.

1. Объявление let-переменной **msg** – глобальной области видимости.
2. Инициализация переменной **msg** текстовым значением "Добро пожаловать!".
3. Переопределение переменной **msg**.
4. Добавление к глобальному лексическому окружению определения функции **fnAuth**().
5. Создание вложенного лексического окружения (объекта Environment Record) при вызове функции:
    - вызов **fnAuth**('Winnie-the-Pooh', 'ilovehoney', 'w-t-p@mail.ru')
    - вызов **fnAuth**('Piglet', 'rainisstarting', 'pig&co@mail.ru').

Обратите внимание, вложенные объекты **Environment Record** (области видимости) создаются в разные периоды времени выполнения программы.

example_16.

```
<script>
    // глобальное лексическое окружение
    // 1
    let msg;
    // 2
    let msg = 'Добро пожаловать!';
    // 3
    msg = 'Посторонним В.';
    // 4
    function fnAuth(login, pwd, email) {
        // область видимости функции
        // локальное (вложенное) лексическое окружение
        // console.log(login, pwd, email)
```



```
    }
    // 5
    // первый вызов функции
    fnAuth('Winnie-the-Pooh', 'ilovehoney', 'w-t-p@mail.ru');
    // второй вызов функции
    fnAuth('Piglet', 'rainisstarting', 'pig&co@mail.ru') ;
    // ----------
    console.log(window);
</script>
```



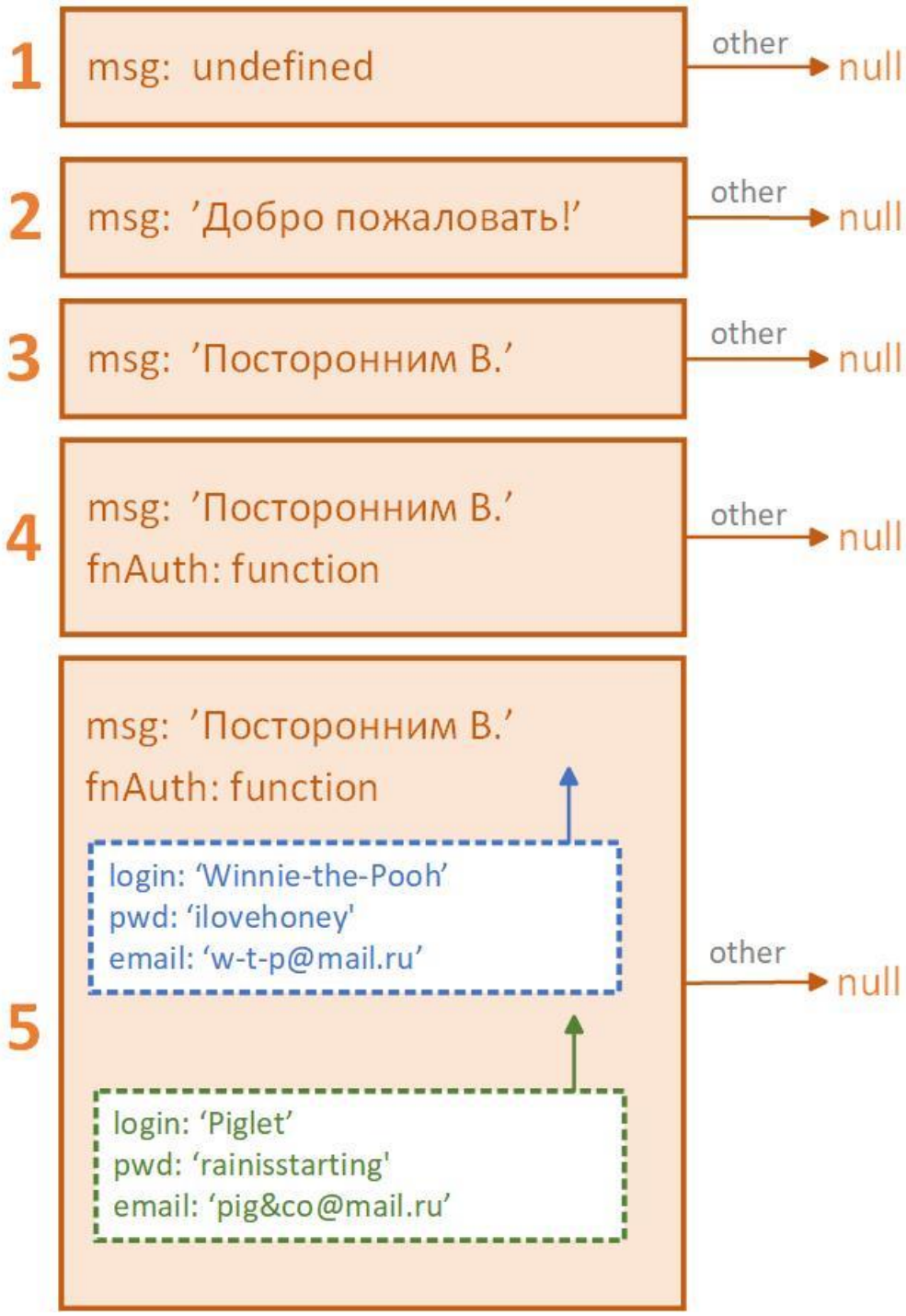
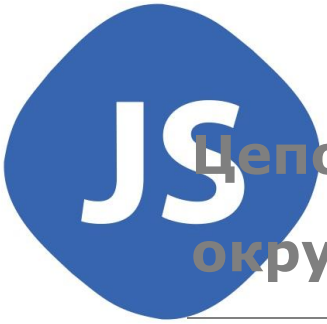


Рис.7. Последовательное формирование локальных лексических окружений



Цепочки объектов лексического окружения

JavaScript поддерживает **вложенные** лексические среды, в простом случае это означает, что функция имеет доступ к переменным и функциям, определенным в **глобальной среде.**

example_17.

```
<script>
    // глобальное лексическое окружение
    var a = 0;
    let b = 5;
    const c = 7;
    // функция глобального лексического окружения
    myFun = () => {
        // область видимости функции
        // локальное (вложенное) лексическое окружение
        // переменная 'a' перезапишет глобальное значение
        a = 10;
        let b = 7;
        // переменная 'b' перезапишет локальное значение
        b = 10;
        console.log(a + b + c); // 10 + 10 + 7 = 27
        // console.log('a = ', a);
        // console.log('b = ', b);
        // console.log('c = ', c);
    }
```



```
    // вызов функции сформирует локальное лексическое окружение
    myFun();
    // ----------
    console.log(window);
</script>
```

В демонстарционном примере **example_17** есть строка кода:

```
// переменная 'a' перезапишет глобальное значение
a = 10;
```

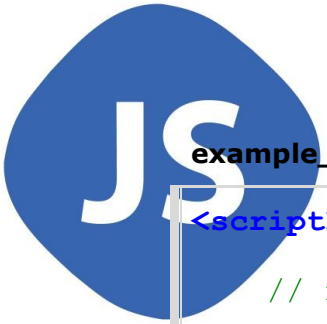
Перед тем, как присвоить значение переменной "a", движок JavaScfript попробует сначала её найти. Поиск переменной всегда начинается с **текущей области видимости**. В приведенном примере ни **переменной** ни **параметра** "a" в области видимости вызова функции **myFun**() нет. Поэтому переходим по ссылке, в данном случае ведущую в **глобальную область видимости** и ищем переменную там.

> ***Важно***. Когда код хочет получить доступ к переменной – сначала происходит поиск во **внутреннем** лексическом окружении, затем во **внешнем**, затем в следующем и так далее, до **глобального**

> ***Примечание***. Поиск переменной в текущем или родительском лексическом окружении зависит от **способа объявления переменной** (let/var/const/ ...).

В более сложном примере поддержка вложенной лексической среды означает, что функция может быть определена внутри другой функции, и эта внутренняя функция будет иметь доступ к переменным и функциям, определенным во внешней функции.

Рассмотрим код следующего демонстрационного примера.



example_18.

```
<script>
    // глобальное лексическое окружение
    var a = 5;
    let b = 10;
    const c = 20;
    // функция глобального лексического окружения
    function fnOuter() {
        // область видимости функции fnOuter
        // локальное (первое вложенное) лексическое окружение
        // переменная 'a' перезапишет глобальное значение
        a = 10;
        let b = 15;
        b = 20;
        // функция лексического окружения созданного fnOuter
        function fnInner() {
            // область видимости функции fnInner
            // локальное (второе вложенное) лексическое окружение
            // переменная 'b' перезапишет значение родительского
            окружения
            b = 2;
            let c = 5;
            console.log(a + b + c); // 10 + 2 + 5 = 17
            // console.log('a = ', a);
            // console.log('b = ', b);
            // console.log('c = ', c);
```



```
        }
        // вызов функции сформирует второе вложенное лексическое
        окружение
        fnInner();
    }
    // вызов функции сформирует первое вложенное лексическое
    окружение
    fnOuter();
</script>
```

В этом коде функция **fnInner**() определена внутри функции **fnOuter**(), которая в свою очередь, принадлежит глобальному окружению.

Таким образом:

- **локальное лексическое окружение** функции **fnInner**() содержит:
  - переменную "**c**";
  - **ссылку** на внешнее лексическое окружение (fnOuter);
- **локальное лексическое окружение** функции **fnOuter**() содержит:
  - переменную "**b**";
  - переменную "**fnInner**";
  - **ссылку** на внешнее лексическое окружение (global);
- **глобальное лексическое окружение** содержит:
  - переменную "**a**";
  - переменную "**b**";
  - переменную "**c**";
  - переменную "**fnOuter**";
  - **ссылку** на внешнее лексическое окружение (null).

Все написанное отображено на следующем рисунке:

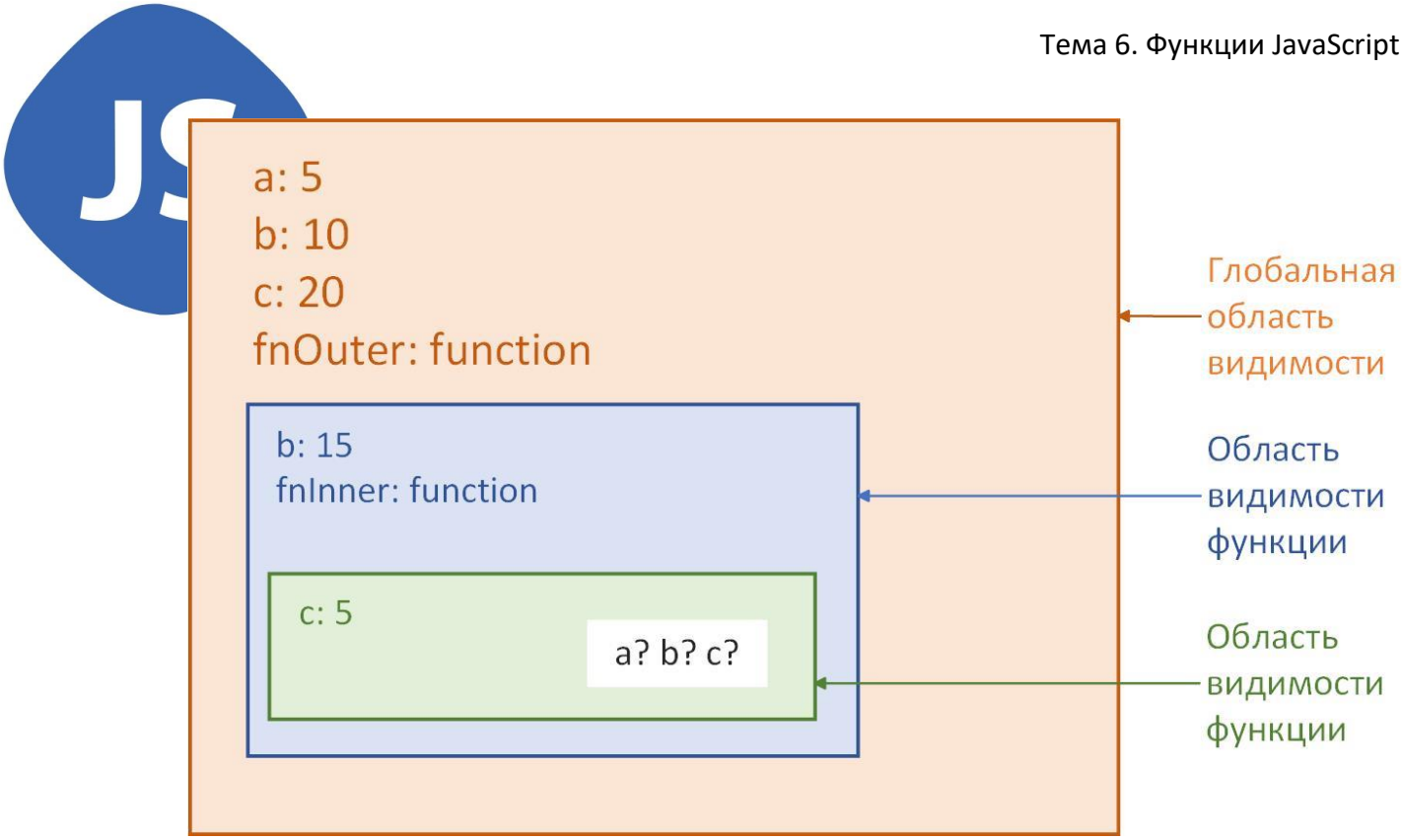


Рис.8. Формирование вложенных лексических окружений

example_19.

```
<script>
    // глобальное лексическое окружение
    let a = 4; // 4
    let c = 2; // 2
    // функция глобального лексического окружения
    function atheEquation(b) {
        // область видимости функции theEquation
        // локальное (первое вложенное) лексическое окружение
        let d = b**2 - 4 * a * c;
        // функция вложенного лексического окружения
        function aroots(d) {
            // область видимости функции roots
            // локальное (второе вложенное) лексическое окружение
```

```
            if (d > 0) {
                let x1 = (-b + Math.sqrt(d)) / (2 * a);
                let x2 = (-b - Math.sqrt(d)) / (2 * a);
                return `x1 = ${x1}; x2 = ${x2}`;
            } else if (d == 0) {
                let x = -b / (2 * a);
                return `x = ${x}`;
            } else {
                return 'нет корней';
            }
        }
        // вызов функции сформирует второе вложенное лексическое
        окружение
        return `d = ${d} \n` + aroots(d);
    }
    // вызов функции сформирует первое вложенное лексическое
    окружение
    console.log(atheEquation(6));
    // ----------
    console.log(window);
</script>
```

example_20.

```
<script>
    // глобальное лексическое окружение
    let a = 5;
```

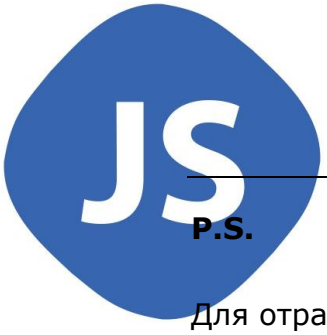
```
    let b = 5;
    let c = 5;
    let obj = {
        // локальное (вложенное) лексическое окружение
        a: 10,
        b: 10,
        c: 10,
        calcSum: function() {
            // метод возвращает сумму трех переменных
            return (a + b + c);
        }
    }
    console.log(obj.calcSum());  // 15
    console.log(a + b + c);  // 15
    console.log(obj.a + obj.b + obj.c);  // 30
</script>
```

Заключение

ФсЁ ...;)

Для закрепления пройденного материала в **самостоятельных работах** выполним следующие задания:

- Поиск ошибки вывода функции в сценарии.
- Математические вычисления и подключаемый скрипт.
- Математические вычисления и подключаемый модуль.
- Математические вычисления и callback функция.



P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы.

